

1 · PIPELINE + BOW

tokenize → normalize (stem/lemma/stop-word/min_df) → vectorize (BoW or TF-IDF) → model	
Concept	Definition
Tokenize	Split text into tokens. Always step #1.
BoW	Vector of word counts. Loses order.
Term-Doc Matrix	Rows=docs, cols=terms, cells=freq/binary.
Stem	Suffix chop, no dict: 'running'→'run'.
Lemmatize	Dict base form: 'feet'→'foot', 'better'→'good'.
Stop-word	Drop 'is','the','a' (low info).
min_df filter	Drop rare tokens (don't generalize).

2 · TF-IDF · N-GRAM · POS

TF-IDF = $(k/n) \cdot \log(N/df)$. k=term count in doc, n=#terms in doc, N=corpus, df=docs containing term.
⇒ 1000 docs, doc has 25 terms, 'unstructured' appears 3× and in 1 of docs (df=200) → **(3·log 5)/25**.
n-grams capture local order: 'not good'≠'good'. Sentence with W tokens → **W-1 bi-grams, W-2 tri-grams**. Ex: 'Social media generates lots of unstructured data' (7) → **6 bi-grams**.
POS labels noun/verb/etc; preserves sentence-level partial order. Higher n + POS ⇒ very high-dim sparse matrix.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer(ngram_range=(1,2), min_df=2)
X = vec.fit_transform(corpus)
from nltk import pos_tag, word_tokenize
pos_tag(word_tokenize(text)) # [(word, tag), ...]
```

3 · SIMILARITY · CLUSTERING · LDA

Method	Note
Cosine sim	angle between vectors; ignores length. Default for text.
Jaccard	$ A \cap B / A \cup B $ (sets/binary).
Overlap	$ A \cap B / \min(A , B)$.
K-Means	Hard 1-cluster/doc, k centroids. Topic = top words of centroid.
Agglomerative	Bottom-up hierarchical merges.
LDA	Probabilistic: doc = mixture of topics; topic = distribution of words.
Cluster vs LDA	Cluster = 1 cluster/doc; LDA = overlapping topic membership.
<pre>from sklearn.cluster import KMeans from sklearn.decomposition import LatentDirichletAllocation as LDA km = KMeans(n_clusters=5).fit(X); lda = LDA(n_components=5).fit(X)</pre>	

4 · SENTIMENT · LOGIT

Sentiment = classify tone (pos/neg/neu). Score = **weighted sum** of word presence × sentiment weight.
VADER = lexicon-based polarity (no training). Adjusts for caps, '!!!', booster/negation, slang. Use sid.polarity_scores(text) → {'neg','neu','pos','compound'}.
Logit: $P = 1/(1+e^{-x})$. β = **per-word sentiment weights** (interpretable). β learned by maximizing log-likelihood on labeled data.
Train/test split mandatory ⇒ tests **generalization**. Without it, accuracy inflated.
ROC: TPR vs FPR over thresholds: trades off recall vs false alarms.

```
from sklearn.linear_model import LogisticRegression
model.fit(train_x, train_c)
y_pred = model.predict(test_x)
acc = accuracy_score(test_c, y_pred) # (true, pred)
```

5 · OTHER CLASSIFIERS

Model	Idea
Naive Bayes	Assumes feature independence; fast, simple baseline.
SVM	Max-margin hyperplane; kernels (poly) → non-linear.
Decision Tree	Recursive splits; interpretable; overfits.
Random Forest	Bag many trees + bootstrap rows/cols → cuts variance/overfit.
Logit	Linear, interpretable β/term.

⇒ RF overfits (train≠test) ⇒ **raise bootstrap subsampling** of features+rows. Don't deepen trees.
Fair contest: same train/test data + same features; compare on **test accuracy**. Overfit when train acc > test acc (model learned noise).

6 · NEURAL NETWORKS + IMAGES

Neuron: $a = f(\sum w_i x_i + b)$. **Activations**: ReLU $\max(0, x)$, sigmoid $1/(1+e^{-x})$. Add non-linearity.
vs Logit/SVM: stacks many **hidden layers** → learns non-linear hierarchical features (low → mid → high).
Param count dominated by **1st layer** (depends on input size). 400×400×3 input → 1 hidden of 480k = **230B params**; 1 hidden of 1k = **480M**. Deep + narrow ■ shallow + wide for same capacity.
⇒ **MLPClassifier(hidden_layer_sizes=(2,3)) = 2 hidden layers: 1st has 2 neurons, 2nd has 3**.
Image: flatten 2D RGB to vector. 100×200×3 → **60000 dims = (60000,1)**. Then any classic ML applies.
Bia nets need bia data — DL beats traditional only at large data size: otherwise overfits.

```
from sklearn.neural_network import MLPClassifier
DL = MLPClassifier(solver='lbfgs', hidden_layer_sizes=(3,2))
DL.fit(train_x, train_c); acc=accuracy_score(test_c, DL.predict(test_x))
```

7 · WORD EMBEDDING

Encoding	Properties
Index	word→int. Preserves order; no semantics.
One-hot	binary vector/word. Orthogonal → no similarity; sparse.
BoW	sum of one-hots; loses order.
N-gram rep	Preserves phrase-level local order.
POS rep	Preserves order within short sentence.
W2V / GloVe	Dense, low-dim; distance = semantic similarity.

⇒ Padded one-hot for K docs, max len L (pad shorter), vocab V → shape **(K, L, V)**. e.g. 123 docs, max 50, V=4000 → **(123, 50, 4000)**.
Semantic arithmetic: king – man + woman = queen. **Pre-trained** (Google W2V, Stanford GloVe) = dict {term: vec}; great for small/noisy data.
Embedding layer in NN = trainable dense vectors learned during training; standard front-end of LSTM/RNN.

8 · RNN + LSTM

Order matters for translation, chatbot, sentiment ⇒ use order-preserving rep (index/one-hot).
RNN: 1 unit reused across time-steps; hidden state carries memory. A step is NOT a unit. Param count depends only on per-step input size ⇒ huge savings vs flattened.
Vanishing/exploding gradient: backprop through long seq fails ⇒ only later tokens learned. **LSTM** adds **memory cell + input/forget/output gates** ⇒ keeps long-range deps; learns earlier tokens too. Built-in attention distribution.
⇒ **Batch size = #samples before weight update (memory)**. **Epoch = 1 full pass**. Need many epochs with batches so early batches re-seen after late batches.
Seq2Seq: produces output sequence (translation. chatbot).

```
model = Sequential([
    Embedding(V, 64), LSTM(32),
    Dense(1, activation='sigmoid')])
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.fit(X, y, batch_size=64, epochs=10)
```

KEY QUIZ TRAPS — TEXT/DL

- Term-doc matrix represents **word frequencies across docs** (not lengths/POS).
- Lemmatization = **dictionary mapping** (not stemming/TF-IDF/n-gram).
- Cosine measures **angle** between vectors (not Jaccard/Euclidean).
- LDA improvement: **overlapping topic membership** per doc (not equal clusters).
- accuracy_score args = **(test_c, y_pred)** = (true, predicted).
- Logit β = **sentiment weights per term** (not tokens/output/embedding).
- Embedding > one-hot because **semantic similarity preserved**.
- LSTM solves **vanishing gradient** (not dim/speed/sparsity).
- RF overfit fix: **bootstrap aggregating** (more random subsample).

9 · WEB SCRAPE (SCRAPY + XPATH)

HTML = nested tags w/ **id** (unique) + **class** (reusable). XPath queries the tree.

XPath / call	Meaning
//div[@class='c']/a	Direct < <a>g< children of any div.c
//div[@class='c']/a	All <a>g< descendants of div.c (any depth)
/div[@class='c']/a	From root only (rarely useful)
.extract()	List of all matches
.extract_first()	First match or None
response.urljoin(rel)	Convert relative href → absolute

Wrapper pattern (multi-feature item): arab parent block. loop. null each child bv relative XPath.

```
def parse(self, response):
    response.xpath('//div[@class="post quote"]'):
    q = w.xpath('span/text()').extract_first()
    a = w.xpath('div[@class="source"]/text()').extract_first()
    yield {'quote': q, 'author': a}
```

Pagination (same-type pages): next href → urljoin → **yield Request(url, callback=self.parse)**.

Hierarchical (different page types): multiple parse fns. Pass partial via Request(..., meta={...}); receive in lower fn via response.meta.

```
next_url = response.urljoin(
    response.xpath('//div[@id="pagination"]/a/@href').extract()[-1])
yield Request(next_url, callback=self.parse)
yield Request(detail_url, callback=self.parse_lower,
    meta={'Quote': q, 'Author': a})
```

☞ *Quiz: only **yield Request(url=..., callback=self.fn)** is correct — capital R, self., no quotes.*

10 · NETWORK BASICS + REPRESENTATION

Network=nodes (vertices) + edges (ties). **Undirected** (FB friend, kinship) vs **Directed** (Twitter follow): tail→head.

Walk=any edge sequence. **Path**=no repeat nodes. **Cycle**=closed path. **Geodesic**=shortest path; geodesic distance = its length. **Component**=maximal connected subgraph. **Strongly connected**=path between every pair (directed).

Representation	Form
Adjacency list	<code>[node: [neighbors]]</code>
Adjacency matrix	N×N binary; symmetric ⇔ undirected
Edge list	<code>[(u,v), ...]</code>

☞ *Max edges: undirected **n(n−1)/2**, directed **n(n−1)**. Directed graph has ½ **density** of its undirected version.*

```
import networkx as nx
G = nx.Graph(adj_list) # from dict
G = nx.from_numpy_array(A) # from matrix
G = nx.Graph(edges) # from edge list
G = nx.read_edgelist('g.edgelist', nodetype=int)
```

11 · NETWORK DIAGNOSIS (NETWORK-LEVEL)

Metric	Formula / meaning
Density	2 E /(V (V −1)) undirected; halved for directed.
Avg degree	Σ deg(v)/ V .
Clustering (transitivity)	closed triplets / triplets (triangles).
Diameter	max shortest-path; undirected only; K-level→2K.
Connectivity	min nodes/edges to disconnect → robustness.
Reciprocity	sym edges / total (directed only).
Centralization	spread of centrality across nodes.

☞ *Same density ≠ same speed: **clustered edges = local groups** (slow spread); dispersed edges spread fast. Same clustering also ≠ same speed.*

```
nx.density(G) nx.transitivity(G)
nx.diameter(G) nx.node_connectivity(G)
nx.edge_connectivity(G) nx.reciprocity(G)
# centralization: max c - all c, sum / ((N-1)(N-2))
```

12 · NODE CENTRALITY (INFLUENCERS)

Centrality	Captures / pick when...
Degree	# direct connections; reach immediate neighbors.
Closeness	1/mean geodesic to all; speed of reach.
Betweenness	# geodesics through node; broker/gatekeeper; cuts flow.
Eigenvector	Connected to other influential nodes (recursive).
Clustering (node)	closed/total triplets in neighborhood; local cohesion.

Structural hole = gap between sub-groups bridged by 1 node ⇒ **high betweenness**; that node controls info flow (broker, competitive advantage).

☞ ***Seeding by goal**: non-contagious post, reach many → **degree**. Reach quickly → **closeness**. Reach other influencers → **eigenvector**. Counter campaign / cut flow → **betweenness**. Reach well-knit subgroups → seed inside them.*

```
nx.degree_centrality(G) nx.closeness_centrality(G)
nx.betweenness_centrality(G) nx.eigenvector_centrality(G)
nx.clustering(G)
top5 = sorted(c.items(), key=lambda x:x[1], reverse=True)[:5]
```

13 · COMMUNITY DETECTION

Method	How / Limit
Components	<code>nx.connected_components(G)</code>; for disconnected communities.
Girvan-Newman	Iteratively remove edge w/ highest betweenness. Heavy compute; not scalable
Stoer-Wagner	Global min cut (max-flow/min-cut); only 2 communities; also gives edge connect
k-Core	Subgraph w/ every node deg ≥ k. Drops sparse nodes; can return 0/1 commun
Louvain	Hierarchical greedy modularity maximization; scalable for large nets.

Louvain process: (1) each node = own community (2) Phase 1: move node to neighbor community to ↑ modularity (3) Phase 2: collapse communities into super-nodes (4) repeat until no improvement.

Modularity Q: density of intra-community edges vs inter. **Q=0.7**=strong, **Q=0.2**=weak. **Pre-partition checks**: cohesion + centralization ⇒ does net look like it has communities?

```
from networkx.algorithms.community import girvan_newman
next(girvan_newman(G)) # first split
cut, parts = nx.stoer_wagner(G) # min cut + 2 parts
core = nx.k_core(G, k=2) # ≥2 connections
import community.community_louvain as cl
parts = cl.best_partition(G) # {node: community}
```

KEY QUIZ TRAPS — WEB/SNA

- //div[@class='content']/a = **direct children only**; // = any descendant.
- Correct request: **yield Request(url='...', callback=self.parse_data)**.
- nx.transitivity / nx.clustering DO NOT identify influential nodes (they measure cohesion).
- Clustering coef of node 2 in triangle {1,2,3,4 path with 1-2,1-3,1-4,2-?} — count closed triplets / total triplets in neighborhood.
- **k-Core** is the only community method that *loses nodes*.
- **Stoer-Wagner** also yields edge connectivity.
- **Louvain** = best for large networks (greedy modularity).
- Modularity Q is the *post-partition* indicator; cohesion+centralization are *pre-partition*.

FAST-RECALL CARDS

BoW vs One-hot: BoW=sum of one-hots, no order. One-hot keeps order, very high-dim.

TF-IDF higher when: common in *this* doc, rare in corpus.

Cosine vs Jaccard: cosine=angle (continuous); Jaccard=set overlap (binary).

K-means vs LDA: hard vs soft assignment; centroid vs distribution.

Logit β: word's contribution to sentiment score (interpretable).

NB / SVM / DT / RF / Logit: compare on same train/test by accuracy.

MLP shape (a,b,c): 3 hidden layers a→b→c neurons.

Vanishing gradient: RNN problem; LSTM gates fix it.

Batch×Epoch: small batch = more updates/epoch; many epochs needed.

Density vs clustering: density=overall edge frac; clustering=triangle frac.

Diameter: longest *shortest* path.

Reciprocity: directed only.

Influencer ranking: degree (popular) · closeness (fast) · betweenness (broker) · eigenvector (well-connected to powerful).